Guía de desarrollo — Daniel

Daniel Alejandro Guevara Rosas · M0 Página mínima publicada · M2 Negocios y categorías · Offero, versión del 27 de octubre de 2026

> Esta guía es tu hoja de ruta para tus módulos: qué construir, en qué orden, con qué tablas y permisos, y cómo saber que quedó bien. No trae código: el código lo escribes tú. Las historias con puntos van en el Product Backlog del plan (Parte 5). Si algo aquí choca con el documento de contexto, manda el documento y le avisas a Diego. Lo marcado con [SUPUESTO] es una propuesta que todavía hay que confirmar.

Tu resumen

Módulos	M0 Página mínima publicada (complejidad baja, ≈10 h) y M2 Negocios y categorías (media, ≈26 h).
Carga estimada	≈36 h de ≈47 h netas disponibles en todo el proyecto.
Rol transversal sugerido	Responsable de pruebas (QA) y dueño del diseño base (layout y estilos) [se confirma en la Parte 2].
Desbloqueas a	Todo el equipo (M0); Wilson (tabla categories); Diego (tabla businesses para cotizar); Iker (página del negocio donde van las reseñas).
Dependes de	Diego: cuentas de Vercel y Supabase, y M1 para saber quién es proveedor.
Fechas clave	M0 publicado **dom 1 nov** · categories con datos reales mar 10 nov · businesses y «Mi negocio» dom 15 nov · M2 terminado dom 22 nov.

> **M0 es lo primero de la ruta crítica.** Si te atascas más de 45 minutos, escribe de inmediato en #ayuda: un día perdido en M0 es un día perdido para los cuatro.

Tu calendario por sprint

Sprint	Qué entregas
0 · 28 oct – 8 nov	Preparar tu computador; M0 publicado el dom 1 nov; probar el flujo de Preview; aprender JS moderno y SQL básico; acompañar a Wilson en su primer PR.
1 · 9 – 15 nov	Tabla categories con categorías reales (mar 10); tabla businesses con RLS; formulario «Mi negocio» (dom 15).
2 · 16 – 22 nov	Listado con filtro y detalle; prueba cruzada; M2 terminado (dom 22).
3 · 23 – 29 nov	Pruebas cruzadas del flujo principal (QA) con usuarios reales; ajustes de diseño.
4 · 30 nov – 6 dic	Integrar las reseñas en el detalle con Iker; probar M5 y M6.
5 · 7 – 11 dic	Estabilización, corrección de errores y presentación.

Ruta de aprendizaje (justo a tiempo)

Aprende cada tema justo antes de la tarea que lo necesita, no todo al principio. El orden importa: cada punto se apoya en el anterior.

Antes de M0 (Sprint 0, primera semana)

1. Terminal de PowerShell: moverte entre carpetas (`cd`, `ls`, `mkdir`).
2. Git: clonar, rama, commit, push y pull request (Anexo C).
3. npm: qué es package.json y qué hacen `npm install` y `npm run dev`.

4. Next.js App Router: cada carpeta dentro de `src/app` es una ruta; `page.js` es la página y `layout.js` el marco común.
5. JSX: cómo se diferencia del HTML que ya sabes (className, etiquetas cerradas, expresiones entre llaves).
6. Tailwind: clases utilitarias y diseño adaptable (prefijos sm:, md:).
7. Vercel: qué es un despliegue y cómo leer el registro de compilación.

Antes de M2 (Sprint 0, segunda semana, y Sprint 1)

1. JavaScript moderno: const y let, funciones flecha, map y filter, desestructuración, async/await.
2. Componentes de React y props.
3. SQL básico: tabla, tipos, clave primaria, clave foránea, UNIQUE, CHECK, SELECT con WHERE y JOIN.
4. Leer datos de Supabase en un componente de servidor, con el cliente que deja Diego.
5. Formularios con Server Actions y validación en el servidor.
6. RLS: qué es una política; diferencia entre USING (qué filas puedes ver o tocar) y WITH CHECK (qué filas puedes crear o dejar).
7. Rutas dinámicas (`[businessId]`) y parámetros de búsqueda (`?categoria=`) para filtros.

Tu rol transversal: pruebas (QA)

- Mantén en `docs/` una lista de verificación por módulo: camino feliz, casos de error y permisos por rol (la Parte 9 del plan la detalla).
- Al cierre de cada sprint, organiza la prueba cruzada: nadie prueba su propio módulo.
- Registra los errores en la sección 10 del documento con este formato: archivos afectados, mensaje de error, pasos para reproducirlo y resultado esperado frente al obtenido.

Módulo 0 — Página mínima publicada

Objetivo	Crear el proyecto Next.js con Tailwind dentro del repositorio App-Offero, publicarlo en Vercel y dejar la base para que el resto del equipo pueda trabajar.
Alcance	Mínimo = completo. Todo M0 es obligatorio y es lo primero de la ruta crítica: hasta que exista el proyecto publicado, nadie más puede programar.
Fuera de alcance	Inicio de sesión, base de datos o cualquier dato (eso es M1 y M2). No pongas enlaces a páginas que todavía no existen.
Depende de	Diego: etiqueta del prototipo, permisos del repositorio y conexión con Vercel. La cuenta de Vercel debe ser la suya, porque es el dueño del repositorio y el plan Hobby solo admite un dueño.
Desbloquea a	A todo el equipo.

Rutas y archivos

- `src/app/page.js` — portada: nombre de Offero y descripción del producto (sección 1 del documento de contexto).
- `src/app/layout.js` — idioma `es` en la etiqueta html, título y descripción de la app, encabezado y pie.
- `src/components/layout/` — `Header.js` y `Footer.js`.
- `src/app/globals.css` — lo que genera Tailwind; si el equipo acuerda colores de marca, se definen aquí.
- `docs/` — la documentación anterior (CONTEXTO, EXPOSICION, CHECKLIST, RUTA.md).
- `README.md`, `.env.example` y `.gitignore`.
- No crees carpetas vacías: Git no las guarda. Cada carpeta aparece cuando tenga su primer archivo.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Pídele a Diego que cree la etiqueta `prototipo-v0` sobre main y que te agregue como colaborador.	mié 28 oct	La etiqueta aparece en GitHub (Tags) y aceptaste la invitación.
2	Prepara tu computador (Anexo B) y clona el repositorio en `C:\dev`.	jue 29 oct	`git status` dentro de `C:\dev\App-Offero` responde sin errores.
3	Crea la rama `chore/m0-proyecto-base`. Mueve CONTEXTO, EXPOSICION, CHECKLIST y RUTA.md a una carpeta `docs/` y borra PROTOTIPOS (queda guardado en la etiqueta). Haz commit.	jue 29 oct	En VS Code, la raíz solo tiene `docs/` y `.git`.
4	Crea el proyecto en una carpeta aparte, porque create-next-app no funciona en una carpeta que ya tiene archivos. En `C:\dev` ejecuta `npx create-next-app@latest offero-base`. Respuestas: TypeScript **No**; linter (ESLint) **Sí** o el valor por defecto; Tailwind **Sí**; carpeta src/ **Sí**; App Router **Sí**; Turbopack y alias de importación: valores por defecto. Si aparece una pregunta nueva, deja el valor por defecto y anótala en el PR.	vie 30 oct	Se crea `C:\dev\offero-base` sin errores.
5	Copia todo el contenido de offero-base a la raíz de App-Offero, **excepto** `node_modules` y `.git`. Luego, dentro de App-Offero: `npm install` y `npm run dev`. Borra offero-base al terminar.	vie 30 oct	http://localhost:3000 muestra la página inicial de Next.js.

#	Paso	Fecha meta	Cómo verificar que quedó bien
6	Arma la página mínima: portada, encabezado, pie, idioma es y metadatos.	sáb 31 oct	Se ve bien en escritorio y en vista de celular (herramientas del navegador).
7	Archivos de base: README con los pasos de instalación en Windows; `.env.example` solo con los nombres de las variables de Supabase, sin valores; revisa `.gitignore`.	sáb 31 oct	`git status` muestra `.env.example` y NO muestra `.env.local` ni `node_modules`.
8	Commit, push y pull request con descripción, capturas y checklist. Revisa Diego.	sáb 31 oct	El PR queda aprobado y fusionado.
9	Con Diego (30 min): él importa App-Offero en su cuenta de Vercel; tú verificas la URL de producción.	dom 1 nov	La URL carga la portada sin errores en la consola.
10	Prueba el flujo de Preview: abre un PR pequeño (por ejemplo, un texto del pie) y confirma que Vercel genera un enlace de vista previa.	mar 3 nov	El PR muestra el enlace de Preview. Si sale «Deployment Blocked», avisa a Diego (riesgo 8 del plan).
11	Actualiza el documento de contexto: enlace del repositorio y de la app (sección 1); M0 Terminado y Publicado: Sí (sección 8).	mar 3 nov	El cambio está fusionado en main.

Ojo con el .gitignore: las versiones recientes de create-next-app ignoran todos los archivos que empiezan por `.env`. Agrega una excepción para `.env.example`; si no, no se subirá y nadie sabrá qué variables necesita.

Criterios de aceptación

- **Dado** cualquier persona, **cuando** abre la URL de producción, **entonces** ve la portada de Offero con estilos y sin errores en la consola.
- **Dado** un integrante con Windows, **cuando** sigue el README desde cero, **entonces** la app corre en localhost:3000 en menos de 15 minutos.
- **Dado** un pull request de cualquier integrante, **cuando** se abre, **entonces** Vercel genera un despliegue de vista previa.
- **Dado** el repositorio, **cuando** alguien revisa main, **entonces** no hay `node_modules`, `.env.local` ni claves.

Errores típicos y cómo evitarlos

- Crear el proyecto dentro de OneDrive: se vuelve lento y aparecen bloqueos. Usa `C:\dev`.
- Elegir TypeScript por error: borra offero-base y repite. El proyecto es JavaScript (decisión del documento).
- Copiar la carpeta `.git` de offero-base: dañaría el historial del repositorio.
- Funciona en tu PC pero falla en Vercel: casi siempre es un import con mayúsculas o minúsculas distintas al nombre real del archivo. Windows no las distingue; Vercel (Linux) sí. Lee el log de compilación.
- Subir `node_modules`: si `git status` lo muestra, detente y revisa el `.gitignore`.

Coordinación con el equipo

- Diego: etiqueta, permisos y conexión con Vercel (pasos 1 y 9).
- Wilson: trabaja contigo en pareja durante M0 para aprender la estructura del proyecto. Explícale cada paso.
- Todos: avisa en Discord en cuanto M0 esté en main, para que cada uno haga `npm install`.

Módulo 2 — Perfiles de negocios y categorías

Objetivo	Los proveedores crean y editan su perfil comercial; cualquier persona consulta los negocios y los filtra por categoría.
Alcance	Mínimo = completo. Todo M2 es obligatorio (Must).
Fuera de alcance	Logos o fotos (Storage; solo si sobra tiempo), mapas, búsqueda por texto, varios negocios por proveedor.
Depende de	Diego (M1): roles, funciones `is_admin()` y `get_my_role()` y el patrón para proteger rutas (listo el sáb 7 nov).
Desbloquea a	Wilson necesita `categories` para las solicitudes; Diego necesita `businesses` para cotizar; Iker muestra reseñas en tu página de detalle.

Tabla `categories`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
name	text	Obligatorio y único.
description	text	Opcional.
created_at	timestamptz	Automático.

Políticas RLS de `categories`

Operación	Quién	Condición
SELECT	Todos, incluso sin sesión	Siempre.
INSERT, UPDATE, DELETE	Admin	`is_admin()`. La pantalla la construye Iker en M7; acuerden quién escribe esta política.

Las categorías iniciales son reales: las define Diego (PO) con lo que salga de las entrevistas en Bogotá. Se cargan con un archivo SQL en dev y en prod.

Tabla `businesses`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
owner_id	uuid → profiles.id	Obligatorio y único: un negocio por proveedor [SUPUESTO].
name	text	Obligatorio, 2 a 100 caracteres.
description	text	Obligatorio, hasta 1000 caracteres.
category_id	uuid → categories.id	Obligatorio. El documento dice `category`; se propone clave foránea para que M7 administre categorías sin romper datos. Registrar la decisión en la sección 11.
location	text	Obligatorio: ciudad y barrio (Bogotá en el piloto).
contact_info	text	Obligatorio: teléfono, WhatsApp o correo comercial.
is_hidden	boolean	Falso por defecto. Solo el admin lo cambia (M7).
created_at	timestamptz	Automático.

Políticas RLS de `businesses`

Operación	Quién	Condición
SELECT	Todos	`is_hidden` es falso; el dueño y el admin ven también los ocultos.
INSERT	Proveedor	Su rol es provider y `owner_id` es el usuario actual.
UPDATE	Dueño	Solo su negocio. No puede cambiar `owner_id` ni `is_hidden`.
DELETE	Nadie en el MVP	El admin oculta en vez de borrar [SUPUESTO].

RLS controla filas, no columnas. Para impedir que el dueño cambie `owner_id` o `is_hidden`, el servidor no envía esos campos y la base lo bloquea con el mismo mecanismo que Diego usa para el rol en profiles. Pídele que te lo muestre.

Rutas y archivos

- `src/app/negocios/page.js` — listado de negocios visibles, con filtro por categoría (`?categoria=`).
- `src/app/negocios/[businessId]/page.js` — detalle: nombre, descripción, categoría, ubicación y contacto; espacio para las reseñas de Iker. Si no existe o está oculto: página 404.
- `src/app/negocios/mi-negocio/page.js` [SUPUESTO: ruta nueva] — crear o editar el negocio del proveedor; si ya tiene uno, carga sus datos.
- `src/components/businesses/` — BusinessCard, BusinessForm y CategoryFilter (Wilson lo reutiliza en solicitudes).
- `src/services/categories.js` y `src/services/businesses.js` — todas las consultas a Supabase de estas tablas, incluida «obtener el negocio del usuario actual», que Diego usa en M4.
- `supabase/sql/` — un archivo para categories, uno para businesses y uno para cargar las categorías reales.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Tabla categories + RLS + carga de categorías reales. Prueba en dev; Diego la aplica en prod al fusionar. Avisa a Wilson.	mar 10 nov	Sin sesión, la consulta de categorías devuelve filas; alguien que no es admin no puede crear.
2	Tabla businesses + RLS en dev.	jue 12 nov	Con dos cuentas de proveedor, cada una crea y edita solo el suyo; un cliente no puede crear.
3	Página «Mi negocio»: formulario con Server Action y validación en el servidor; solo para proveedores.	dom 15 nov	Un proveedor crea su negocio; si ya tiene uno, el formulario lo edita.
4	Listado, tarjeta y filtro por categoría.	mié 18 nov	El filtro cambia la URL y muestra solo esa categoría.
5	Detalle del negocio; 404 si no existe o está oculto.	vie 20 nov	Un id inventado en la URL muestra la página 404.
6	Enlace «Mi negocio» en el encabezado, solo para proveedores (coordina con Diego, que pone el menú de sesión).	sáb 21 nov	Un cliente no ve el enlace y tampoco puede abrir la ruta.
7	Prueba cruzada (la hace Wilson) y publicación con negocios reales de las empresas del piloto.	dom 22 nov	Checklist de la Definition of Done completo (Anexo G).

Criterios de aceptación

- **Dado** un proveedor sin negocio, **cuando** completa nombre, descripción, categoría, ubicación y contacto y guarda, **entonces** su negocio aparece en /negocios y al filtrar por su categoría.
- **Dado** un proveedor que ya tiene negocio, **cuando** intenta crear otro, **entonces** el sistema no lo permite y le muestra el formulario de edición.
- **Dado** un cliente, **cuando** intenta abrir /negocios/mi-negocio o crear un negocio directo contra Supabase, **entonces** es redirigido o la operación falla.
- **Dado** el proveedor A, **cuando** intenta editar el negocio del proveedor B cambiando el id, **entonces** la operación falla.
- **Dado** un visitante sin sesión, **cuando** filtra por una categoría, **entonces** ve solo los negocios visibles de esa categoría.

Pruebas de permisos

Prueba como visitante, cliente, proveedor A, proveedor B y admin estas operaciones: ver la lista, ver el detalle, crear, editar el propio y editar el ajeno. Anota el resultado esperado y el obtenido.

Errores típicos y cómo evitarlos

- Proteger solo ocultando botones: la seguridad está en el servidor y en RLS.
- Olvidar la política SELECT pública: la lista sale vacía para los visitantes sin ningún mensaje de error.
- Validar solo en el formulario del navegador.
- No revisar el error que devuelve cada consulta a Supabase.
- La lista no se actualiza después de guardar: hay que revalidar la ruta.

Coordinación con el equipo

- Wilson: categories debe estar el mar 10 nov; compártele CategoryFilter.
- Diego: expón en tu servicio una función para obtener el negocio del usuario actual; M4 la usa para saber desde qué negocio se cotiza.
- Iker: en el Sprint 4 coloca en tu página de detalle sus componentes RatingSummary, ReviewList y ReportButton. Él escribe la política de admin para ocultar negocios.
- Como dueño del layout, cualquier cambio de otros en `layout.js`, `globals.css` o `components/layout/` pasa por tu revisión.

Anexo — Acuerdos comunes del equipo

Este anexo es igual en las cuatro guías. Si cambia, se cambia en todas.

A. Calendario del equipo

Sprint	Fechas	Meta del equipo
0	mié 28 oct – dom 8 nov	Proyecto publicado (M0), entornos listos, todos con su computador preparado, M1 en marcha
1	lun 9 – dom 15 nov	M1 terminado; tablas categories, businesses, requests y reports creadas con RLS
2	lun 16 – dom 22 nov	M2 y M3 terminados; envío de cotizaciones; panel de admin básico
3	lun 23 – dom 29 nov	M4 terminado: solicitud → cotización → elegir, demostrable con usuarios reales
4	lun 30 nov – dom 6 dic	M5 y M6 (mínimos) terminados; M7 mínimo terminado
5	lun 7 – vie 11 dic	Estabilización: pruebas, errores y presentación. Entrega: vie 11 dic

Festivos en Colombia dentro del periodo: lun 2 nov, lun 16 nov y mar 8 dic.

B. Preparar tu computador (Windows, PowerShell)

#	Paso	Comando o acción	Cómo verificar
1	Instalar Node.js LTS	`winget install OpenJS.NodeJS.LTS` (o el instalador LTS de nodejs.org)	`node -v` y `npm -v` muestran versión
2	Instalar Git	`winget install Git.Git`	`git --version`
3	Permitir scripts de npm	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responde S)	`npx -v` funciona sin el error «scripts deshabilitados»

4	Configurar Git	`git config --global user.name "Tu Nombre"`
		`git config --global user.email "correo de tu GitHub"`
		`git config --global init.defaultBranch main`
			`git config --global --list`

5	Extensiones de VS Code	ESLint, Tailwind CSS IntelliSense y Prettier; opcional: GitLens	Aparecen en la pestaña Extensiones

6	Clonar fuera de OneDrive	`mkdir C:\dev`
		`cd C:\dev`
		`git clone https://github.com/diegodani2030/App-Offero.git`
			Existe `C:\dev\App-Offero`

7	Instalar dependencias (cuando M0 esté en main)	`cd C:\dev\App-Offero` `npm install`	Termina sin errores en rojo
8	Variables de entorno	`Copy-Item .env.example .env.local` y llena los valores que Diego te envía por mensaje privado	`.env.local` existe y `git status` no lo muestra
9	Correr la app	`npm run dev` y abre http://localhost:3000	Ves la portada de Offero

Abre una PowerShell nueva después de instalar Node y Git. El correo de Git debe ser el de tu cuenta de GitHub. La primera vez que hagas push, Windows abrirá el navegador para iniciar sesión en GitHub.

C. Flujo de Git paso a paso

#	Qué haces	Comando o acción
1	Toma una tarea del tablero y muévela a «En curso»	GitHub Projects
2	Actualiza main	`git switch main` → `git pull`
3	Crea tu rama: tipo/mN-descripcion (tipos: feat, fix, chore, docs)	`git switch -c feat/m3-formulario-solicitud`
4	Guarda avances pequeños con mensajes en español: tipo(módulo): qué hace	`git add .` → `git commit -m "feat(m3): agrega formulario de nueva solicitud"`
5	Sube la rama	`git push -u origin feat/m3-formulario-solicitud` (después, solo `git push`)
6	Abre el pull request hacia main con la plantilla: qué cambia, cómo probarlo, capturas y checklist. Escribe «Cierra #N» para enlazar la tarea	GitHub
7	Revisión: al menos un compañero. Si toca base de datos, autenticación o archivos compartidos, revisa Diego. Corrige con nuevos commits en la misma rama	GitHub
8	Fusión: la hace Diego con «Create a merge commit» (por la restricción de Vercel Hobby); luego se borra la rama	GitHub
9	Actualiza tu rama con main cada día	`git switch main` → `git pull` → `git switch tu-rama` → `git merge main`
10	Si hay conflicto, VS Code muestra las opciones (cambio actual, entrante o ambos). Si no entiendes el otro cambio, pregúntale a su autor. Luego guarda y haz commit	`git add .` → `git commit`

- Nunca hagas commits directos a main ni `git push --force`.
- Pull requests de menos de un día de trabajo: son más fáciles de revisar y casi no generan conflictos.
- No subas `.env.local`, `node_modules` ni claves. Si `git status` los muestra, detente y pregunta.

D. Convenciones de base de datos

- **Dos proyectos de Supabase** [SUPUESTO; se confirma en la Parte 3]: `offero-dev` para desarrollo y pruebas (ahí sí puedes crear datos de prueba) y `offero-prod` solo con datos reales (lo usa la versión de producción en Vercel). Tu `.env.local` y las previews apuntan a dev.
- **Cada cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql` (por ejemplo `20261111_1530_requests.sql`). La fecha y hora evitan choques entre cuatro personas. Se prueba en dev, va en el PR y Diego lo aplica en prod al fusionar. Nunca edites un archivo ya aplicado: crea uno nuevo. Nadie cambia tablas desde el panel sin dejar su archivo.
- **Todas las tablas**: id uuid automático (profiles usa el id de Auth), created_at automático, claves foráneas explícitas y restricciones CHECK para estados y números.
- **RLS se activa en el mismo archivo que crea la tabla.** Una tabla sin políticas no se fusiona.
- **Contenido moderable** (businesses, requests, reviews) incluye `is_hidden` desde su creación, para que M7 no tenga que modificar tablas ajenas [SUPUESTO; registrar en la sección 11 del documento].
- **Funciones de ayuda de Diego**: `is_admin()` y `get_my_role()`. Úsenlas en las políticas en vez de repetir consultas.

- **Nombres de personas**: siempre desde la vista `public_profiles`. Los correos nunca se muestran a otros usuarios.
- **Validación en tres capas**: formulario (comodidad), servidor de Next.js (seguridad) y base de datos (restricciones y RLS, la última barrera).

Tabla.campo	Valores acordados (fuente única)
`profiles.role`	client, provider, admin
`requests.status`	open (recibe cotizaciones), awarded (adjudicada), cancelled
`quotes.status`	pending, accepted, rejected, withdrawn
`reports.status`	open, reviewed, dismissed
`reports.target_type`	business, request, review

E. Archivos compartidos

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisa en Discord antes de cambiarlos; cambios pequeños
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/`	Dueño de cada tabla; aprueba Diego	No editar archivos ya aplicados
`package.json` y `package-lock.json`	Todos, con acuerdo	No instales paquetes sin acordarlo (restricción de stack). Si hay conflicto en el lock: acepta la versión de main y vuelve a ejecutar `npm install`
`docs/` (documento de contexto)	Cada uno, su módulo	Actualiza las secciones 8, 9 y 10 al avanzar
`src/components/ui/`	Quien crea el componente	Si cambias un componente compartido (ReportButton, CategoryFilter), avisa

F. Comunicación, regla de atasco e IA

- **Canales** [SUPUESTO]: Discord para el trabajo (#daily para el avance diario, #ayuda para bloqueos, #prs para pedir revisión). WhatsApp solo para urgencias.
- **Regla de atasco**: después de 45 minutos intentando solo, escribe en #ayuda qué intentabas, el archivo, el mensaje de error completo y qué probaste. Si en 2 horas nadie responde, menciona a Diego. Un bloqueo de más de un día se dice en la Daily.
- **Revisiones**: responde las que te pidan en menos de 24 horas.
- **IA**: pega el documento de contexto al iniciar cada conversación; pide explicaciones, no solo código; verifica que use JavaScript, App Router y `@supabase/ssr` (no TypeScript, ni la carpeta `pages/`, ni Flask); no subas nada que no puedas explicar en la revisión; nunca pegues claves en la IA.

G. Definition of Done

Una historia está hecha cuando:

- cumple sus criterios de aceptación;
- su pull request fue revisado y fusionado;
- funciona en la Preview y luego en producción;
- sus permisos se probaron con dos cuentas de roles distintos;

- no deja errores en la consola del navegador;
- el documento de contexto refleja cualquier cambio.

Un módulo está Terminado cuando (regla del documento de contexto):

- funciona en la versión publicada en Vercel;
- sus operaciones principales están probadas por otra persona (prueba cruzada);
- no depende de datos ficticios en producción;
- la sección 8 del documento dice Terminado y Publicado: Sí.